PMLZA4. Bayesian computation

UoP

DataLab

August 2026

ICMAT-CSIC

Contents

Chapter objectives

Objectives

- Recap on Bayesian analysis methods
- Identify tough computational problems that PML faces
- Describe how to solve them (with not much math detail...)
- Some practicalities about the solutions
- (Long) Lab including Stan
- As an aside: Intro to some models not seen yet

Motivation

PML Setup

Observed data: x

Model: $p(x \mid \theta)$

Unknown (ie uncertain) parameter: θ

Prior distribution: $p(\theta)$

First goal: Update beliefs about θ after observing x

Posterior Distribution

Using Bayes' rule $p(\theta \mid x) = \frac{p(x|\theta)\, p(\theta)}{p(x)}$

where the marginal likelihood (evidence) is

$$p(x) = \int p(x \mid \theta)\, p(\theta)\, d\theta$$

Key idea:

Posterior $\propto$ Likelihood $\times$ Prior

Posterior Predictive Distribution

Goal: Predict new observation x_{new}

$$p(x_{\text{new}} \mid x) = \int p(x_{\text{new}} \mid \theta)\, p(\theta \mid x)\, d\theta$$

Interpretation:

- Average predictions over uncertainty in θ
- Incorporates both model and parameter uncertainty

Bayesian decision analysis setup

Goal: Choose between actions a whose impact is affected by θ

Utility function assesses preferences and risk attitudes of decision maker

$$u(a, \theta)$$

Choose action maximizing posterior expected utility after having observed data x

$$\mathbb{E}[u(a, \theta) \mid x] = \int u(a, \theta) \, p(\theta \mid x) \, d\theta$$

Optimal action

$$a^* = \arg\max_a \int u(a, \theta) \, p(\theta \mid x) \, d\theta$$

Maximize predictive expected utility

MC and MCMC integration

Monte Carlo Integration Setup

We want to compute an integral over θ:

$$I = \int f(\theta)\, p(\theta)\, d\theta$$

Interpretation:

$$I = \mathbb{E}_{p(\theta)}[f(\theta)]$$

Challenge

- Integral often intractable (high dimension and/or complex $p(\theta)$)...

Monte Carlo Approximation

Draw samples

$$\theta^{(1)}, \ldots, \theta^{(N)} \sim p(\theta)$$

and replace expectation with sample average

Approximate the integral by

$$I \approx \frac{1}{N} \sum_{i=1}^{N} f(\theta^{(i)})$$

- Accuracy improves as $N \to \infty$
- Probabilistic bounds of error O($\sqrt{n}$)

Application in Bayesian Inference

Example: posterior expectation

$$\mathbb{E}[f(\theta) \mid x] = \int f(\theta)\, p(\theta \mid x)\, d\theta$$

Monte Carlo estimate

$$\mathbb{E}[f(\theta) \mid x] \approx \frac{1}{N} \sum_{i=1}^{N} f(\theta^{(i)}), \quad \theta^{(i)} \sim p(\theta \mid x)$$

Used for

- Posterior means and probabilities
- Predictive distributions
- Decision-making (expected utility)

Why MCMC?

We want to compute $I = \int f(\theta)\, p(\theta|x)\, d\theta$

Problem:

- Direct sampling from $p(\theta|x)$ may be difficult
- $p(\theta|x)$ typically may only be known up to a constant:

$$p(\theta|x) \propto \tilde{p}(\theta)$$

MCMC idea

- Construct a Markov chain whose stationary distribution is $p(\theta|x)$
- Sample from Mchain until convergence, then collect (and process sample), then approximate expectations by sample means

MCMC Approximation

Generate a Markov chain

$$\theta^{(1)}, \theta^{(2)}, \ldots, \theta^{(N)}$$

such that (after burn-in)

$$\theta^{(i)} \sim p(\theta)$$

Approximate

$$I = \int f(\theta)\, p(\theta)\, d\theta \;\approx\; \frac{1}{N} \sum_{i=1}^{N} f\left(\theta^{(i)}\right)$$

- Samples are *dependent*
- Still converges under mild conditions

GS, MH, Hybrid and HMC samplers

Gibbs sampler. Gibbs for kids

(X,Y) Bernoulli variables with joint distribution

X	Y	$P(X,Y)$
0	0	p_1
1	0	p_2
0	1	p_3
1	1	p_4

Compute the marginals of X and Y

Compute the conditionals

The conditionals are characterised by

$$A_{yx} = \begin{pmatrix} P(Y=0|X=0) & P(Y=1|X=0) \\ P(Y=0|X=1) & P(Y=1|X=1) \end{pmatrix} = \begin{pmatrix} \dfrac{p_1}{p_1+p_3} & \dfrac{p_3}{p_1+p_3} \\ \dfrac{p_2}{p_2+p_4} & \dfrac{p_4}{p_2+p_4} \end{pmatrix}$$

$$A_{xy} = \begin{pmatrix} \dfrac{p_1}{p_1+p_2} & \dfrac{p_2}{p_1+p_2} \\ \dfrac{p_3}{p_3+p_4} & \dfrac{p_4}{p_3+p_4} \end{pmatrix}$$

Gibbs sampler. Gibbs for kids

Set $Y_0 = y_0$, $i = 1$

repeat

    Generate $X_i \sim X \mid Y = y_{i-1}$

    Generate $Y_i \sim Y \mid X = X_i$

    Set $i = i + 1$

until stopping criterion is satisfied

Gibbs sampler. Gibbs for kids

A Markov chain with transition matrix

$$A = A_{xy} A_{yx}$$

Gibbs sampler. Gibbs for kids

$$(p_1 + p_3 \;\; p_2 + p_4) = (p_1 + p_3 \;\; p_2 + p_4)A$$

$X_n \to^d X$

$Y_n \to^d Y \; (X_n, Y_n) \to^d (X, Y)$

Gibbs Sampler

Given $\theta = (\theta_1, \ldots, \theta_d)$ obtain the conditionals of θ_i given the other variables. Initialise the variables arbitrarily at $t = 0$

Iteratively sample from full conditionals, until convergence (and the sample until desired size)

$$\theta_1^{(t+1)} \sim p(\theta_1 \mid \theta_2^{(t)}, \ldots, \theta_d^{(t)})$$
$$\theta_2^{(t+1)} \sim p(\theta_2 \mid \theta_1^{(t+1)}, \theta_3^{(t)}, \ldots, \theta_d^{(t)})$$
$$\cdots$$
$$\theta_d^{(t+1)} \sim p(\theta_d \mid \theta_1^{(t+1)}, \ldots, \theta_{d-1}^{(t+1)})$$

Example

$$\pi(x_1, x_2) = \frac{1}{\pi} e^{-x_1(1+x_2^2)} \qquad (x_1, x_2) \in (0, \infty) \times (-\infty, \infty)$$

$$\pi(x_1|x_2) = \frac{\pi(x_1, x_2)}{\pi(x_2)} \propto \pi(x_1, x_2) \propto e^{-x_1(1+x_2^2)} \qquad X_1|X_2 = x_2 \sim \mathcal{E}xp(1 + x_2^2)$$

$$\pi(x_2|x_1) \propto \pi(x_1, x_2) \propto e^{-x_1 x_2^2}, \qquad X_2|X_1 = x_1 \sim \mathcal{N}\left(0, \sigma^2 = \frac{1}{2x_1}\right)$$

Gibbs sampler. Example

Choose the initial value X_2^0

Set $i = 1$

repeat

  Generate $X_1^i \sim \mathrm{Exp}\left(1 + (X_2^{i-1})^2\right)$

  Generate $X_2^i \sim \mathcal{N}\left(0, \frac{1}{2X_1^i}\right)$

  Set $i = i + 1$

until stopping criterion is satisfied

Convergence of Gibbs Sampler

Kernel $$p_G(\theta^n, \theta^{n+1}) = \prod_{i=1}^k p(\theta_i^{n+1} \mid \theta_j^n, j > i; \theta_j^{n+1}, j < i).$$

Proposition 43 *Suppose that* $D = \{\theta : p(\theta) > 0\}$ *is a product set,* $D = \prod_{i=1}^k D_i$. *Then:*

1. $p_{\theta_i}(\theta_i|\theta_j, j \neq i)$ *and* p_G *are well-defined for* $\theta \in D$.
2. p_G *is p-irreducible and aperiodic.*
3. p *is invariant with respect to* p_G.
4. $\theta^n \xrightarrow{w} \theta$

Gibbs for a Mixture Exponentials

- Modeling with mixtures. Density estimation, Clustering,...
- Machine learning. Latent Dirichlet allocation, Generative modeling,...
- Latent variables
- Gibbs sampler

Gibbs for a Mixture of Exponentials

$$f(t \mid \theta) = q_1\mu_1 \exp(-\mu_1 t) + \cdots + q_k\mu_k \exp(-\mu_k t) \qquad \mathbf{t} = (t_1, \ldots, t_{n_s})$$

$$\theta = (q_1, \mu_1; \ldots; q_k, \mu_k), \ \sum q_i = 1, \ q_i > 0. \qquad \mu_i \sim \mathcal{G}(a_i, p_i) \quad \text{and} \quad \mathbf{q} \sim \mathcal{D}(\alpha_1, \ldots, \alpha_k)$$

$$\mathbf{z}_j = (z_{1j}, \ldots, z_{kj}) \qquad \mathbf{z}_j \mid \mathbf{q}, \boldsymbol{\mu} \sim \mathcal{M}_k(1; q_1, \ldots, q_k).$$

$$t_j \mid \mathbf{z}_j, \mathbf{q}, \boldsymbol{\mu} \sim \mathcal{E}\left(\prod_{i=1}^{k} \mu_i^{z_{ij}}\right)$$

$$\mathbf{z}_j \mid t_j, \mathbf{q}, \boldsymbol{\mu} \sim \mathcal{M}_k\left(1; \frac{q_1\mu_1 \exp(-\mu_1 t_j)}{\sum_{i=1}^{k} q_i\mu_i \exp(-\mu_i t_j)}; \ldots; \frac{q_k\mu_k \exp(-\mu_k t_j)}{\sum_{i=1}^{k} q_i\mu_i \exp(-\mu_i t_j)}\right),$$
$$j = 1, \ldots, n_s,$$

$$\mu_j \mid \mathbf{t}, \mathbf{z} \sim \mathcal{G}\left(a_j + \sum_{i=1}^{n} z_{ji} t_i, p_j + \sum_{i=1}^{n} z_{ji}\right), \quad j = 1, \ldots, k,$$

$$\mathbf{q} \mid \mathbf{t}, \mathbf{z} \sim \mathcal{D}\left(\alpha_1 + \sum_{i=1}^{n} z_{1i}, \ldots, \alpha_k + \sum_{i=1}^{n} z_{ki}\right).$$

Gibbs for a Mixture of Exponentials

Start with arbitrary values $(\mathbf{q}^0, \boldsymbol{\mu}^0, \mathbf{z}^0)$, $i = 0$.

Iterate until convergence:

- Generate $\mathbf{z}_j^{i+1} \sim z_j \mid \mathbf{t}, \mathbf{q}^i, \boldsymbol{\mu}^i, \quad j = 1, \ldots, n_s$.
- Generate $\mathbf{q}^{i+1} \sim \mathbf{q} \mid \mathbf{t}, \mathbf{z}^{i+1}$.
- Generate $\mu_j^{i+1} \sim \mu_j \mid \mathbf{t}, \mathbf{z}^{i+1}, \quad j = 1, \ldots, k$.
- Set $i = i + 1$.

Gibbs for Simple Hierarchical Regression Model

Model

$$\mathbf{y} \mid \boldsymbol{\beta}, \sigma^2 \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \sigma^2 \mathbf{I}_n),$$
$$\boldsymbol{\beta} \mid \sigma^2, \tau^2 \sim \mathcal{N}(\mathbf{0}, \sigma^2 \tau^2 \mathbf{I}_k).$$

Hyperpriors

$$\sigma^2 \sim \text{Inverse-Gamma}(a_0, b_0),$$
$$\tau^2 \sim \text{Inverse-Gamma}(c_0, d_0).$$

Joint Posterior

$$p(\boldsymbol{\beta}, \sigma^2, \tau^2 \mid \mathbf{y}) \propto p(\mathbf{y} \mid \boldsymbol{\beta}, \sigma^2)\, p(\boldsymbol{\beta} \mid \sigma^2, \tau^2)\, p(\sigma^2)\, p(\tau^2).$$

Gibbs for Simple Hierarchical Regression Model

Full Conditional Posteriors

Posterior of $\boldsymbol{\beta} \mid \sigma^2, \tau^2, \mathbf{y}$:

$$\boldsymbol{\beta} \mid \sigma^2, \tau^2, \mathbf{y} \sim \mathcal{N}(\boldsymbol{\beta}_n, \sigma^2 \mathbf{V}_n),$$

where

$$\mathbf{V}_n = (\mathbf{X}^\top \mathbf{X} + \tau^{-2} \mathbf{I}_k)^{-1},$$
$$\boldsymbol{\beta}_n = \mathbf{V}_n \mathbf{X}^\top \mathbf{y}.$$

Posterior of $\sigma^2 \mid \boldsymbol{\beta}, \tau^2, \mathbf{y}$:

$$\sigma^2 \mid \boldsymbol{\beta}, \tau^2, \mathbf{y} \sim \text{Inverse-Gamma}\left(a_0 + \frac{n+k}{2},\ b_0 + \frac{1}{2}\left[(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \frac{1}{\tau^2} \boldsymbol{\beta}^\top \boldsymbol{\beta}\right]\right).$$

Posterior of $\tau^2 \mid \boldsymbol{\beta}, \sigma^2$:

$$\tau^2 \mid \boldsymbol{\beta}, \sigma^2 \sim \text{Inverse-Gamma}\left(c_0 + \frac{k}{2},\ d_0 + \frac{1}{2\sigma^2} \boldsymbol{\beta}^\top \boldsymbol{\beta}\right).$$

Gibbs. Comments

- Requires availability and efficent sampling from marginal conditionals... Not always possible....

Metropolis Hastings

Choose a candidate generation distribution $q(.,.)$

Given current state $\theta^{(t)}$

1. Propose

$$\theta' \sim q(\theta' \mid \theta^{(t)})$$

2. Accept with probability

$$\alpha = \min\left(1,\ \frac{p(\theta')\, q(\theta^{(t)} \mid \theta')}{p(\theta^{(t)})\, q(\theta' \mid \theta^{(t)})}\right)$$

3. Update

$$\theta^{(t+1)} = \begin{cases} \theta' & \text{with probability } \alpha \\ \theta^{(t)} & \text{otherwise} \end{cases}$$

Samples from $p(\theta)$ without knowing normalization constant!!!!

Metropolis Hastings: particular cases:

- Random walk chain $q(x, y) = q(x - y)$
- Metropolis Algorithm $q(x, y)$ symmetric (e.g. Normal)
- Independence chain $q(x, y) = q(y)$
- Exploiting form of target

Acceptance rate not too low, not too high.

MH convergence

MH algo defines a Markov process with transition probability

$$p_{\mathrm{MH}}(\theta^n, \theta^{n+1}) = \begin{cases} q(\theta^n, \theta^{n+1})\alpha(\theta^n, \theta^{n+1}), & \theta^n \neq \theta^{n+1}, \\ 1 - \int q(\theta^n, z)\alpha(\theta^n, z)\, dz, & \text{otherwise.} \end{cases}$$

$p_\Theta(\cdot)$ is invariant for this process. Therefore, it is enough to ensure aperiodicity and p_Θ-irreducibility of $p_{\mathrm{MH}}(\cdot)$ to obtain $\theta^n \xrightarrow{w} \theta$.

Proposition

1. If q is aperiodic, then p_{MH} is aperiodic.
2. If q is p_{MH}-irreducible and

$$q(\theta^n, \theta^{n+1}) = 0 \iff q(\theta^{n+1}, \theta^n) = 0,$$

then p_{MH} is p_Θ-irreducible.

MH for logistic regression in bioassay. Setup

- k dose levels: $x_1, \dots, x_k$
- At dose x_i:
  - n_i individuals
  - y_i successes
- Goal: model dose–response relationship

Model

$$Y_i \sim \text{Binomial}(n_i, p_i)$$

Logistic regression:

$$\text{logit}(p_i) = \beta_0 + \beta_1 x_i$$

$$p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_i)}}$$

Likelihood

$$L(\beta_0, \beta_1) = \prod_{i=1}^{k} \binom{n_i}{y_i} p_i^{y_i} (1 - p_i)^{n_i - y_i}$$

Log-likelihood:

$$\ell(\beta_0, \beta_1) \propto \sum_{i=1}^{k} \left[y_i(\beta_0 + \beta_1 x_i) - n_i \log(1 + e^{\beta_0 + \beta_1 x_i}) \right]$$

MLE

Score equations:

$$\sum_{i=1}^{k}(y_i - n_i p_i) = 0$$

$$\sum_{i=1}^{k} x_i(y_i - n_i p_i) = 0$$

- No closed-form solution
- Solve numerically (Newton-Raphson,...)

Bayesian Model

Prior:

$$\boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

Posterior:

$$\pi(\boldsymbol{\beta} \mid \mathbf{y}) \propto L(\boldsymbol{\beta}) \cdot p(\boldsymbol{\beta})$$

Log-Posterior

$$\log \pi(\boldsymbol{\beta} \mid \mathbf{y}) \propto \sum_{i=1}^{k} \left[y_i(\beta_0 + \beta_1 x_i) - n_i \log(1 + e^{\beta_0 + \beta_1 x_i}) \right]$$
$$- \frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\mu})^T \Sigma^{-1} (\boldsymbol{\beta} - \boldsymbol{\mu})$$

Metropolis–Hastings

Proposal:

$$\boldsymbol{\beta}^* \sim \mathcal{N}(\boldsymbol{\beta}^{(t)}, \Sigma_q)$$

Acceptance probability:

$$\alpha = \min\left(1, \frac{\pi(\boldsymbol{\beta}^* \mid \mathbf{y})}{\pi(\boldsymbol{\beta}^{(t)} \mid \mathbf{y})}\right)$$

Log Acceptance Ratio

$$\log \alpha = \log \pi(\boldsymbol{\beta}^* \mid \mathbf{y}) - \log \pi(\boldsymbol{\beta}^{(t)} \mid \mathbf{y})$$

Accept if:

$$\log u < \log \alpha, \quad u \sim \text{Uniform}(0, 1)$$

Algorithm

1. Initialize $\boldsymbol{\beta}^{(0)}$
2. For $t = 0, \ldots, T-1$:
   - Propose $\boldsymbol{\beta}^*$
   - Compute $\log \alpha$
   - Accept/reject
3. Collect samples after burn-in

Posterior Prediction

For new dose x^*:

$$p^* = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x^*)}}$$

$$Y^* \sim \text{Binomial}(n^*, p^*)$$

- Average over posterior samples

Hybrid algorithms

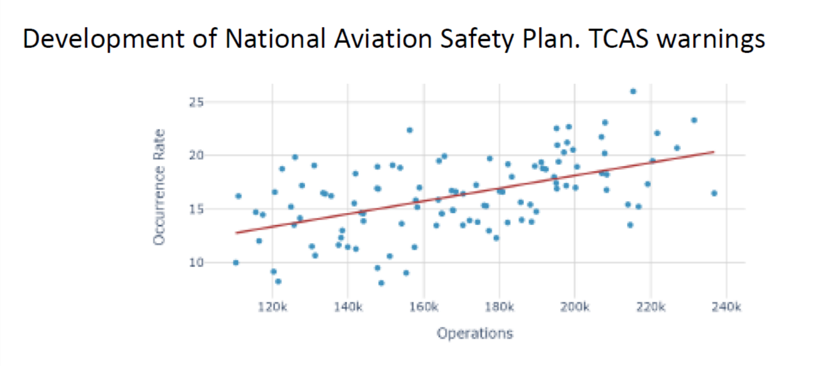
Frequently

1. Some posterior conditionals available for efficient sampling
2. For others, just known up to a constant

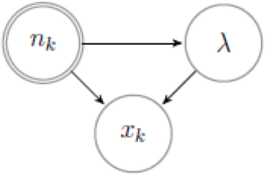
Hybrid samplers

- Gibbs steps for type 1 conditionals
- MH steps for type 2 conditionals

Hybrid sampler within the National Aviation Safety Plan



Hybrid sampler within NASP. Stress effect



$$x_k|\lambda, n_k \sim Po(\lambda n_k),$$

$$\lambda = a n_k + b + \epsilon_k, \quad \epsilon_k \sim N(0, \sigma^2),$$

$$a \sim N(\mu_a, \sigma_a^2), \quad b \sim N(\mu_b, \sigma_b^2), \quad \sigma^2 \sim \textit{Inv-Gamma}(\alpha, \beta)$$

Hybrid sampler within NASP. Stress effect. Conditionals

Posterior proportional to

$$\lambda^{\sum_{i=1}^{k-1} x_i} \sigma^{-3-2\alpha} \exp\left(-\lambda \sum_{i=1}^{k-1} n_i - \frac{(\lambda - a n_k - b)^2}{2\sigma^2} - \frac{(a-\mu_a)^2}{2\sigma_a^2} - \frac{(b-\mu_b)^2}{2\sigma_b^2} - \frac{\beta}{\sigma^2} \right)$$

Posterior conditionals

$$\begin{aligned}
p(a|\lambda, b, \sigma^2, D_k) &\sim N\left(a \,\middle|\, \frac{\sigma^2\mu_a + n_k\sigma_a^2(\lambda - b)}{\sigma^2 + n_k\sigma_a^2}, \frac{\sigma^2}{n_k^2 + \sigma^2/\sigma_a^2}\right), \\
p(b|\lambda, a, \sigma^2, D_k) &\sim N\left(b \,\middle|\, \frac{\sigma^2\mu_b + \sigma_b^2(\lambda - a n_k)}{\sigma^2 + \sigma_b^2}, \frac{1}{1/\sigma_b^2 + 1/\sigma^2}\right), \\
p(\sigma^2|\lambda, a, b, D_k) &\sim \textit{Inv-Gamma}\left(\sigma^2 \,\middle|\, \alpha + \frac{1}{2}, \beta + \frac{1}{2}(b + a n_k - \lambda)^2\right), \\
p(\lambda|a, b, \sigma^2, D_k) &\propto \lambda^{\sum x_i} \exp\left(-\frac{1}{2\sigma^2}\left(\lambda^2 - 2\lambda\left(a n_k + b - \sigma^2 \sum n_i\right)\right)\right)
\end{aligned}$$

Hybrid sampler within NASP. Stress effect. Algo

Algorithm 1: MCMC sampler for *Stress Effect* model

Set $a_0, b_0, \lambda_0, \sigma_0^2, j = 1$;

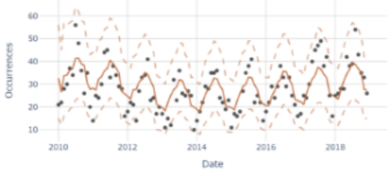
while *convergence not detected* **do**

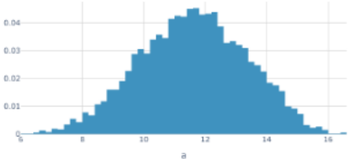
- Sample $\sigma_j^2 \sim \textit{Inv-Gamma}\left(\alpha + \frac{1}{2}, \beta + \frac{1}{2}(b_{j-1} + a_{j-1}n_k - \lambda_{j-1})^2\right)$;
- Sample $b_j \sim N\left(\frac{\sigma_j^2 \mu_b + \sigma_b^2(\lambda_{j-1} - a_{j-1}n_k)}{\sigma_j^2 + \sigma_b^2}, \frac{1}{1/\sigma_b^2 + 1/\sigma_j^2}\right)$;
- Sample $a_j \sim N\left(\frac{\sigma_j^2 \mu_a + n_k \sigma_a^2(\lambda_{j-1} - b_j)}{\sigma_j^2 + n_k \sigma_a^2}, \frac{\sigma_j^2}{n_k^2 + \sigma_j^2/\sigma_a^2}\right)$;
- Sample $\lambda_j^* \sim q(\lambda_{j-1})$; $\quad Ga(1 + \sum_{i=1}^{k-1} x_i, \frac{\lambda}{2\sigma^2} + \sum_{i=1}^{k-1} n_i)$
- Calculate $\alpha = \min\left(1, \frac{\pi(\lambda_j^*)}{\pi(\lambda_{j-1})} \frac{q(\lambda_{j-1})}{q(\lambda_j^*)}\right)$;
- Do $\lambda_j = \begin{cases} \lambda_j^* & \text{with probability } \alpha \\ \lambda_{j-1} & \text{with probability } (1-\alpha) \end{cases}$;
- $j \leftarrow j + 1$;

Hybrid sampler within NASP. Stress effect. Use

$$Pr(x_k = z|D_k) = \iiiint Pr(x_k = z|\lambda, n_k)\, p(\lambda|a, b, \sigma^2, D_k)\, p(a, b, \sigma^2|D_k)\, d\lambda\, da\, db\, d\sigma^2$$

$$\approx \frac{1}{N}\sum_{j=1}^{N} Pr(x_k = z|\lambda_j, n_k) = \frac{n_k^z}{N z!}\sum_{j=1}^{N} \exp(-\lambda_j n_k)\,(\lambda_j)^z\,,$$





Hybrid sampler for NNs

$$y = \sum_{j=1}^{m} \beta_j \psi(x'\gamma_j) + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

$$\psi(\eta) = \exp(\eta)/(1 + \exp(\eta))$$

$$\beta_i \sim N(\mu_\beta, \sigma_\beta^2) \text{ and } \gamma_i \sim N(\mu_\gamma, S_\gamma^2)$$

$$\mu_\beta \sim N(a_\beta, A_\beta),\ \mu_\gamma \sim N(a_\gamma, A_\gamma),\ \sigma_\beta^{-2} \sim \ Gamma(c_b/2, c_b C_b/2)$$

$$S_\gamma^{-1} \sim Wish(c_\gamma, (c_\gamma C_\gamma)^{-1}) \text{ and } \sigma^{-2} \sim Gamma(s/2, sS/2)$$

Start with arbitrary (β, γ, ν).
while *not convergence* **do**
  Given current (γ, ν), draw β from $p(\beta|\gamma, \nu, y)$ (a multivariate normal).
  for $j = 1, \ldots, m$, *marginalizing in* β *and given* ν **do**
    Generate a candidate $\tilde{\gamma}_j \sim g_j(\gamma_j)$.
    Compute $a(\gamma_j, \tilde{\gamma}_j) = \min\left(1, \frac{p(D|\tilde{\gamma}, \nu)}{p(D|\gamma, \nu)}\right)$ with $\tilde{\gamma} = (\gamma_1, \gamma_2, \ldots, \tilde{\gamma}_i, \ldots, \gamma_m)$.
    With probability $a(\gamma_j, \tilde{\gamma}_j)$ replace γ_j by $\tilde{\gamma}_j$. If not, preserve γ_j.
  end
  Given β and γ, replace ν based on their posterior conditionals:
  $p(\mu_\beta|\beta, \sigma_\beta)$ is normal; $p(\mu_\gamma|\gamma, S_\gamma)$, multivariate normal; $p(\sigma_\beta^{-2}|\beta, \mu_\beta)$, Gamma; $p(S_\gamma^{-1}|\gamma, \mu_\gamma)$, Wishart; $p(\sigma^{-2}|\beta, \gamma, y)$, Gamma.
end

Motivation for Hamiltonian Monte Carlo

We want to sample from:

$$p(\theta)$$

Problem with earlier MCMC (GS, MH)

- Random walk behavior (slow exploration)
- High correlation between samples

Idea of HMC:

- A guided proposal generation scheme
- Uses the gradient of log posterior to direct MC towards HPD regions: A well-tuned HMC accepts proposals at much higher rate than MH
- But still samples the tails properly

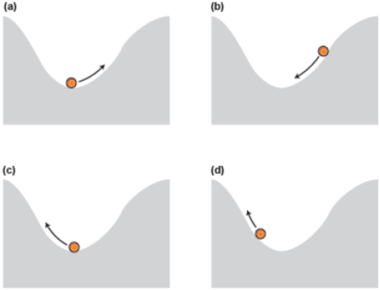
f is the posterior

-log (f) inverse bell-shaped

lower values reached guided by its gradient

In classical mechanics, exchanges between kinetic and potential energy dictate location through hamiltonian equations

(θ, p) horizontal and vertical positions. p is a momentum (auxiliary variable to actually simulate from θ



Augmented Distribution

Introduce momentum:

$$r \sim \mathcal{N}(0, M)$$

Define joint distribution:

$$p(\theta, r) = p(\theta)\, p(r)$$

Hamiltonian function:

$$H(\theta, r) = -\log p(\theta) + \frac{1}{2} r^\top M^{-1} r$$

Interpretation:

- Potential energy: $-\log p(\theta)$
- Kinetic energy: $\frac{1}{2} r^\top M^{-1} r$

Hamiltonian Dynamics

Evolve (θ, r) using:

$$\frac{d\theta}{dt} = \nabla_r H(\theta, r) = M^{-1} r$$

$$\frac{dr}{dt} = -\nabla_\theta H(\theta, r) = \nabla_\theta \log p(\theta)$$

Key properties:

- Moves along high-probability regions
- Avoids random walk behavior

In practice: simulate with leapfrog steps

HMC Algorithm

At iteration t:

1. Sample momentum:

$$r^{(t)} \sim \mathcal{N}(0, M)$$

2. Propose (θ', r'):

   - Simulate Hamiltonian dynamics (leapfrog)

3. Accept with probability:

$$\alpha = \min\left(1,\ \exp(-H(\theta', r') + H(\theta^{(t)}, r^{(t)}))\right)$$

4. Keep θ' if accepted

Leapfrog Integration and Tuning

Leapfrog parameters:

- Step size: ϵ
- Number of steps: L

Starting from (θ, r), repeat:

$$r \leftarrow r + \frac{\epsilon}{2} \nabla_\theta \log p(\theta)$$

$$\theta \leftarrow \theta + \epsilon M^{-1} r$$

$$r \leftarrow r + \frac{\epsilon}{2} \nabla_\theta \log p(\theta)$$

After L steps: obtain proposal (θ', r')

Tuning intuition:

- Small ϵ: accurate but slow exploration
- Large ϵ: faster but lower acceptance

HMC

Tune parameters difficult. But now there's the NUTS algorithm doing it automatically

Providing the gradient of the log posterior may be complex and error-prone... now done automatically

See Lab with Stan

GS;MH,HMC

- Gibbs. Lots of hard prior work. Not always implementable. If so may work well.
- Metropolis Hastings. Less prior work. Quite general. May work slowly.
- Hamiltonian. Even less work. Quite general. Works more efficiently.... Yet suffers in large scale problems

Other samplers

- Reversible jumper (varying dimension problems)
- Particle filtering (dynamic problems, incl state space models)
- Stochastic Gradient MCMC (large scale problems: Big Data, High dimensional)

Some common MCMC themes

Once convergence detected, collect samples from posterior and perform inference (point estimates, intervals, hypothesis tests, predictions and expected utility computations) via Monte Carlo (recall uncertainty associated, they are stochastic algos!!!)

Some common MCMC themes: Difficulties

If iterations have not proceeded long enough, target is not approximated well, samples are unrepresentative of target!!!

Early iterations may bias results

Autocorrelation impacts precision of estimates and the effective number of samples may be smaller than the one actually drawn (as if we'd be using a smaller number of samples)

Some common MCMC themes: Difficulties

Runs to allow for effective monitoring of convergence (based on multiple chains, recall labs)

Monitor convergence by comparing variation within and between simulated sequences (until within and between variation are similar)

Modifying the algorithm by reparameterising or learning good parameterisations, if efficiency is very low (algo too slow)

Discarding initial values

Thinning

Take into account AC when estimating precisions

There are rigorous approaches to all these!!!!

VI

What's up with this algo in big data regimes?

1. Start with arbitrary values $(\mathbf{q}^0, \mu^0, \mathbf{z}^0)$, $i = 0$.

   Iterate until convergence.

2. Generate $\mathbf{z}_j^{i+1} \sim \mathbf{z}_j \mid t_j, \mathbf{q}^i, \mu^i$, $j = 1, \ldots, n_s$.
3. Generate $\mathbf{q}^{i+1} \sim \mathbf{q} \mid \mathbf{t}, \mathbf{z}^{i+1}$.
4. Generate $\mu_j^{i+1} \sim \mu_j \mid \mathbf{t}, \mathbf{z}^{i+1}$, $j = 1, \ldots, k$.
5. Set $i = i + 1$.

What's up with this algo in big data regimes and deep architectures?

Start with arbitrary (β, γ, ν).

while *not convergence* **do**

  Given current (γ, ν), draw β from $p(\beta|\gamma, \nu, y)$ (a multivariate normal).

  for *$j = 1, \ldots, m$, marginalizing in β and given ν* **do**

    Generate a candidate $\tilde{\gamma}_j \sim g_j(\gamma_j)$.

    Compute $a(\gamma_j, \tilde{\gamma}_j) = \min\left(1, \frac{p(D|\tilde{\gamma}, \nu)}{p(D|\gamma, \nu)}\right)$ with $\tilde{\gamma} = (\gamma_1, \gamma_2, \ldots, \tilde{\gamma}_i, \ldots, \gamma_m)$.

    With probability $a(\gamma_j, \tilde{\gamma}_j)$ replace γ_j by $\tilde{\gamma}_j$. If not, preserve γ_j.

  end

  Given β and γ, replace ν based on their posterior conditionals:

  $p(\mu_\beta|\beta, \sigma_\beta)$ is normal; $p(\mu_\gamma|\gamma, S_\gamma)$, multivariate normal; $p(\sigma_\beta^{-2}|\beta, \mu_\beta)$, Gamma; $p(S_\gamma^{-1}|\gamma, \mu_\gamma)$, Wishart; $p(\sigma^{-2}|\beta, \gamma, y)$, Gamma.

end

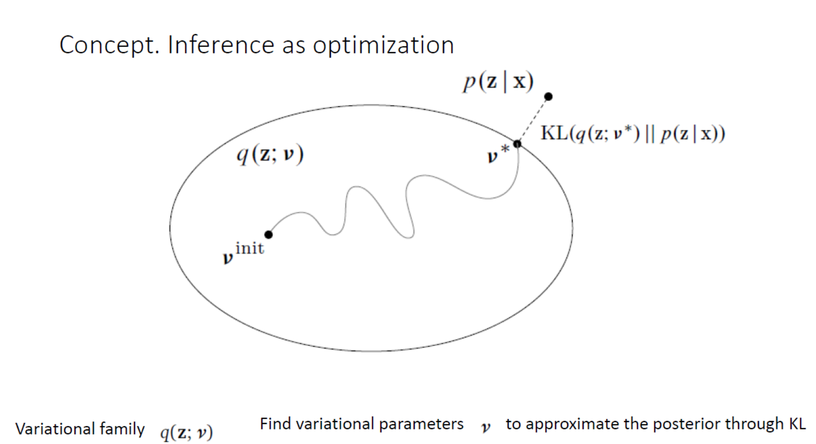
Variational Inference

- Converts inference problem into one of optimization
- Faster than faster SG-MCMC (as we know it today)
- Scales better for large data sets (as we know it today)
- May underestimate uncertainty!!!

Variational Inference. Concept

- Start with probabilistic model of observed variables **x** and latent variables **z (labels and pars)** $p(\mathbf{z}, \mathbf{x})$
- Want to estimate $p(\mathbf{z}|\mathbf{x})$ but difficult because of p(x)
- Approximate with a distribution of efficient computation $q(\mathbf{z})$
- Minimise distance so that they resemble
- Use Kullback-Leibler divergence $$KL(q||p) = \int q(\mathbf{z}) \log \frac{q(\mathbf{z})}{p(\mathbf{z}|\mathbf{x})} d\mathbf{z}$$

Disimilarity measure
Non negative
0 iff they coincid


Concept. Inference as optimization
$p(\mathbf{z} \mid \mathbf{x})$
$\mathrm{KL}(q(\mathbf{z}; \boldsymbol{\nu}^*) \,||\, p(\mathbf{z} \mid \mathbf{x}))$
$q(\mathbf{z}; \boldsymbol{\nu})$
$\boldsymbol{\nu}^*$
$\boldsymbol{\nu}^{\text{init}}$
Variational family $q(\mathbf{z}; \boldsymbol{\nu})$
Find variational parameters $\boldsymbol{\nu}$ to approximate the posterior through KL

Variational Inference. Concept. ELBO

But direct KL optimization impossible!!

$$KL(q||p) \equiv \mathbb{E}_q[\log q(\mathbf{z})] - \mathbb{E}_q[\log p(\mathbf{x},\mathbf{z})] + \log p(\mathbf{x})$$

We may remove last term. Evidence lower bound

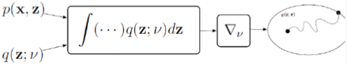
$$ELBO(q) = \mathbb{E}_q[\log p(\mathbf{x},\mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})]$$

Besides $\log p(\mathbf{x}) \geq ELBO(q)$

$$q^\star(\mathbf{z}) = \arg\min_{q\in\mathcal{Q}} KL(q(\mathbf{z})||p(\mathbf{z}|\mathbf{x}))$$

$$\longrightarrow q^\star(\mathbf{z}) = \arg\max_{q\in\mathcal{Q}} ELBO(q)$$

VI recipe



Formulate model $p(\mathbf{z},\mathbf{x})$

Choose variational app $q(\mathbf{z};\boldsymbol{\nu})$

Write and assess ELBO $\mathscr{L}(\boldsymbol{\nu}) = \mathbb{E}_{q(\mathbf{z};\boldsymbol{\nu})}[\log p(\mathbf{x},\mathbf{z}) - \log q(\mathbf{z};\boldsymbol{\nu})]$

Compute gradient $\nabla_{\boldsymbol{\nu}}\mathscr{L}(\boldsymbol{\nu})$

Optimize $\boldsymbol{\nu}_{t+1} = \boldsymbol{\nu}_t + \rho_t \nabla_{\boldsymbol{\nu}}\mathscr{L}$

Stochastic VI

Input:

$\mathcal{D}$, Dataset; $q_\phi(z \mid x)$, Inference model, $p_\theta(x, z)$, Generative model

Result: θ, ϕ: Learned parameters

$(\theta, \phi) \leftarrow$ Initialize parameters

while SGD not converged **do**

- $\mathcal{M} \sim \mathcal{D}$ (random minibatch of data)
- $\epsilon \sim p(\epsilon)$ (random noise for every datapoint in $\mathcal{M}$)
- Estimate $\widehat{\mathcal{L}}_{\theta,\phi}(\mathcal{M}, \epsilon)$ and $\nabla_{\theta,\phi}\widehat{\mathcal{L}}_{\theta,\phi}(\mathcal{M}, \epsilon)$
- Update θ and ϕ using SGD optimizer

end

Final comments

Final comments

- Non-availability of posterior is the norm
- MCMC as core strategy to sample from the posterior
- SG-MCMC and VI for large scale problems with pros and cons
- Hybrids of VI and MCMC as solution
- PPPLs. Probabilistic Programming Languages
- Systems with VI: Venture, WebPPL, Edward, Stan, PyMC3, Infer.net, Anglican,...
- Autodiff tools: Theano, Torch, Tensorflow, Stan, Caffe,...
- Probabilistic Machine Learning

References

References

Gelman, Carlin, et al (2014) Bayesian Data Analysis.

French, Ríos Insua (2000) Statistical Decision Theory.

Murphy (2023) Probabilistic Machine Learning

Probabilistic Machine Learning

Lab4

Labs with various strategies: MC vc MCMC, Gibbs, Metropolis, HMC, Stan (including VB)